MULTIMEDIA UNIVERSITY, MALAYSIA

Operating System Vulnerabilities

ETHICAL HACKING AND PENETRATION TESTING
CCS6324 - 2530

Dr. Hafiz Adnan Hussain

DISCLAIMER

- This course is designed to educate students on ethical hacking principles. The knowledge and techniques acquired must be used responsibly and not for any illegal or unauthorized activities.
- Each student is solely responsible for their actions.
- Do not investigate individuals, websites, servers or conduct any illegal activities on any system you do not have permission to analyze.
- The course content has been developed in good faith, drawing from the author's experience and reputable sources. While every effort has been made to ensure accuracy, the author does not guarantee absolute correctness. Students are encouraged to exercise discretion and actively engage in discussions by raising any questions or concerns during the course.

Learning Objectives

What You Will Learn?

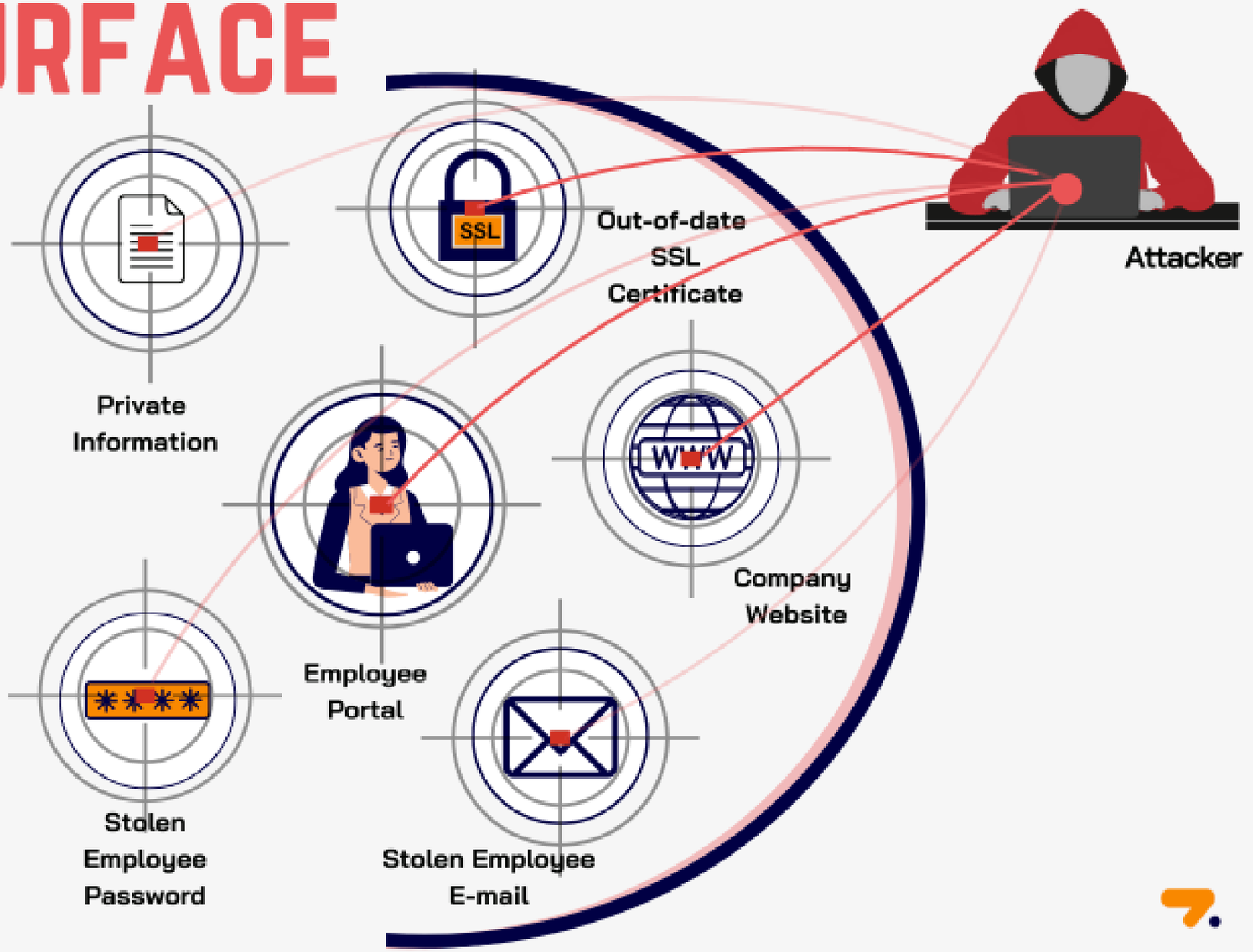
- Identify and analyze OS-specific vulnerabilities in Windows, Linux, and embedded systems
- Understand privilege escalation techniques and exploitation methods
- Apply hardening measures to protect operating systems
- Detect and mitigate rootkit threats
- Assess vulnerabilities in IoT and embedded devices
- Implement countermeasures against OS attacks

Attack Surface

Attack surface is known as the possible points where an unauthorized person can exploit the system with vulnerabilities. It's the combination of weak endpoints of software, system, or a network that attackers can penetrate.

Depending on various parameters and data types, attack surfaces are classified into three types:

1. **Digital** attack surface
2. **Physical** attack surface
3. **Social Engineering** Attack Surface



zenarmor

Windows Operating System Vulnerabilities

1.1 Overview: Operating system vulnerabilities are weaknesses in the OS that allow attackers to perform unauthorized actions. Windows systems remain high-value targets for attackers due to their prevalence in enterprise environments. Vulnerabilities in Windows can be exploited through known **Common Vulnerabilities and Exposures (CVEs)**, **Misconfigurations**, and **Zero-day exploits**.

1.2 Windows File Systems:

File Allocation Table (FAT):

- Original Microsoft file system, supported by nearly all OSs
- Most serious shortcoming: lacks file-level Access Control Lists (ACLs)
- Cannot set permissions on individual files in multiuser environments
- Creates vulnerabilities in shared system access

NTFS (New Technology File System):

- First released as a high-end file system
- Added support for larger files, disk volumes, and ACL file security
- Feature: Alternate Data Streams (ADSs) - ability to hide information behind existing files
- Intruders can hide exploitation tools and malicious files without affecting the file function or size
- Detection methods available: Microsoft STREAMS.EXE

Note: There are many interesting visualizations available on the Internet. It's better to search both terms on Google Images for detailed information on FAT and NTFS.

Windows Operating System Vulnerabilities (Cont'd)

1.3 Common Windows Vulnerabilities

Unquoted Service Path:

When a Windows service has an executable path without quotes and contains spaces, Windows attempts to execute files at different path intervals:

Example: C:\Program Files\Dummy Service\svc01.exe

Windows tries to execute:

- C:\Program.exe
- C:\Program Files\Dummy.exe
- C:\Program Files\Dummy Service\svc01.exe

If an attacker can place a malicious Program.exe in C:\, the service will execute the attacker's code with service privileges (often SYSTEM).

Mitigation:

- Always enclose executable paths in double quotes
- Audit service configurations regularly
- Monitor for unusual service behavior and command-line executions

Insecure Permissions:

Services, files, and registry keys with overly permissive access controls allow low-privilege users to modify critical system components.

Vulnerable Groups:

- Everyone
- BUILTIN\Users
- NT AUTHORITY\Authenticated Users

Low-privilege users may exploit these permissions to:

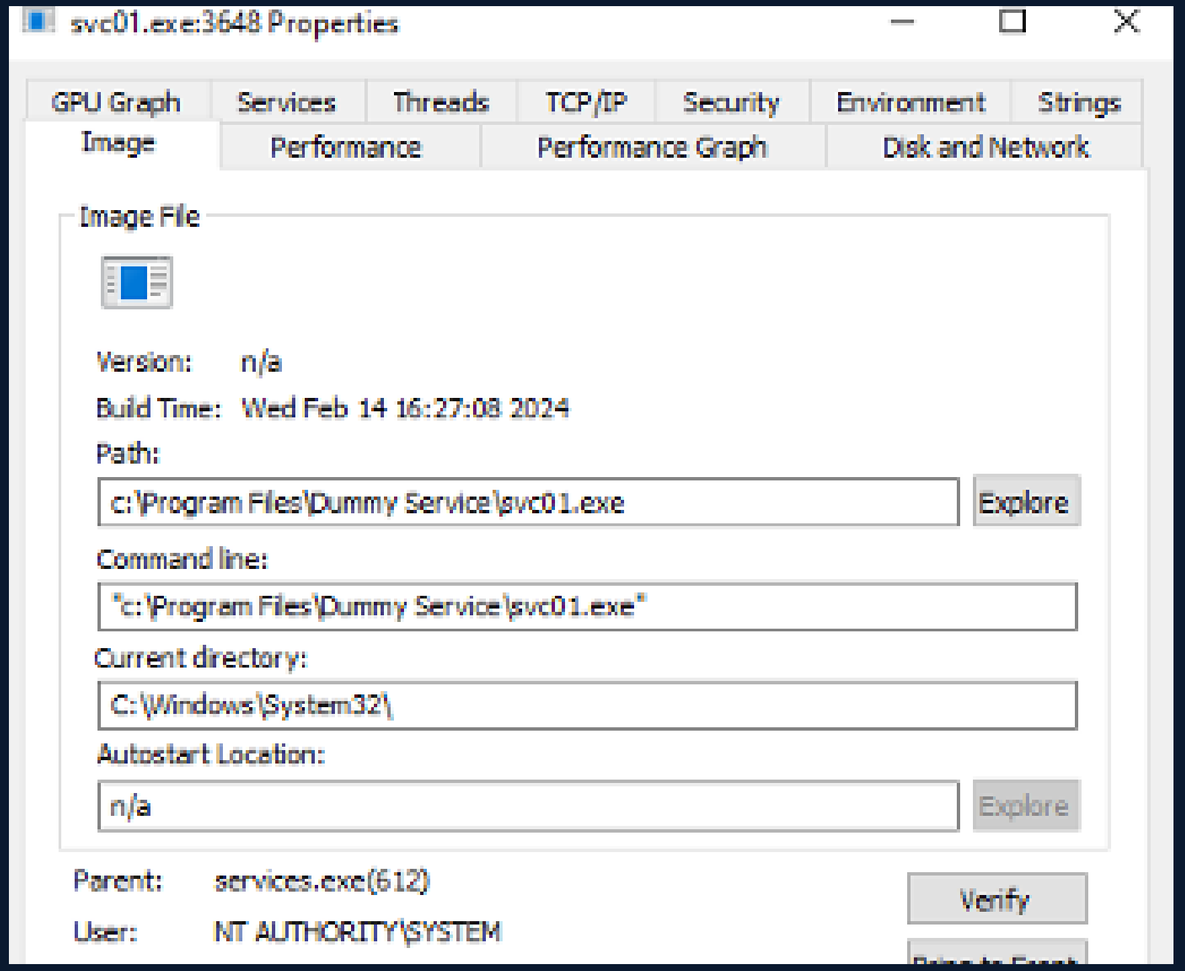
- Write malicious DLLs to system directories
- Modify service executables
- Alter startup configurations

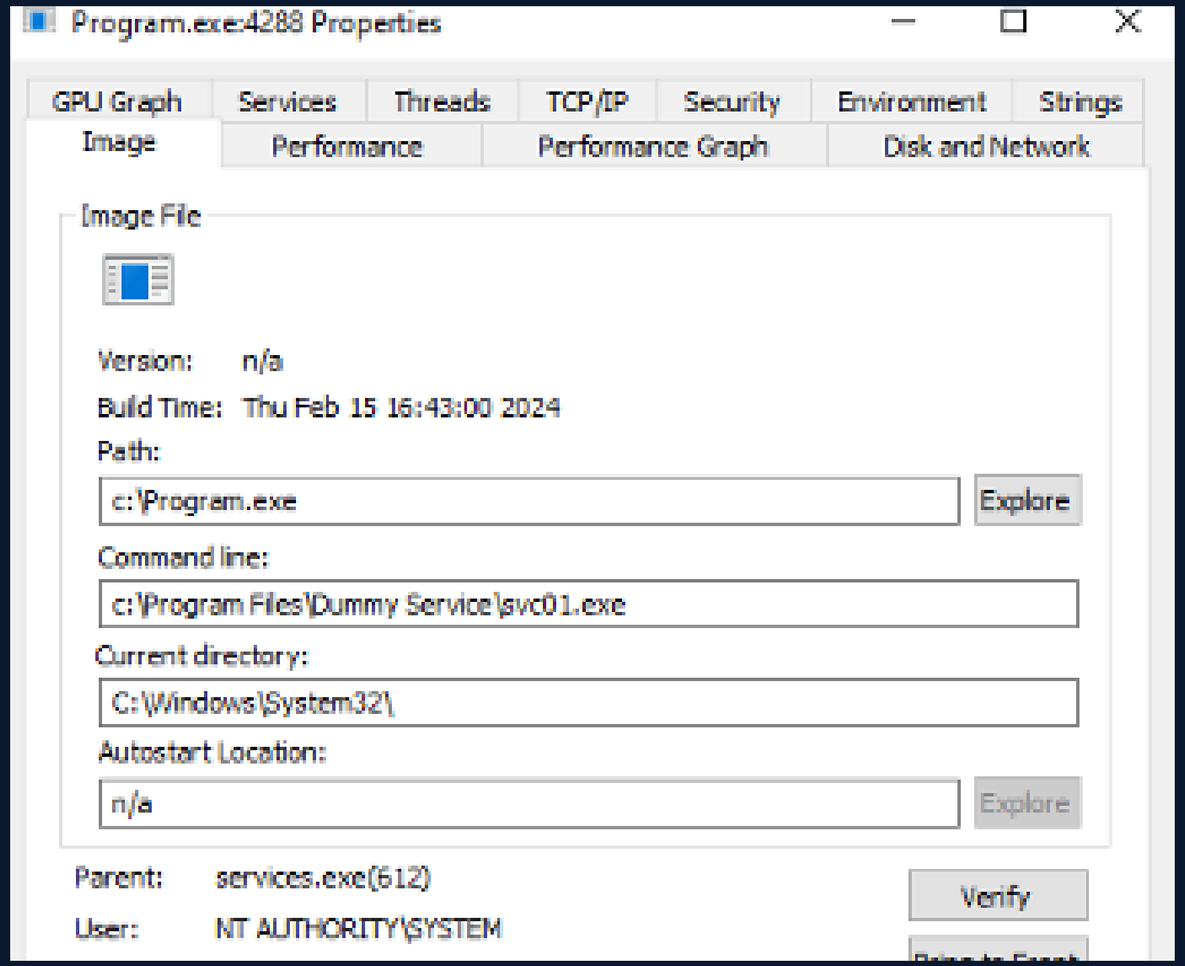
Mitigation:

- Apply least privilege principles to all file and registry permissions
- Regularly audit Access Control Lists (ACLs)
- Use tools like AccessChk to identify insecure permissions
- Implement Group Policy Object (GPO) restrictions

Windows Operating System Vulnerabilities (Cont'd)

An Example of Unquoted Service Path:





Windows Operating System Vulnerabilities (Cont'd)

1.3 Common Windows Vulnerabilities (Cont'd)

System PATH Hijacking:

The Windows **PATH** environment variable specifies directories where executable files are searched. If a directory in the **PATH** is writable by low-privilege users, attackers can place malicious executables there.

Attack Scenario: If C:\CustomProgram is user-writable and appears in **PATH** before C:\Windows\System32, an attacker can create a malicious net.exe in C:\CustomProgram. When an administrator runs the net command, the attacker's version executes with elevated privileges.

Mitigation:

- Restrict write permissions on all **PATH** directories
- Avoid allowing low-privilege users to write to C:\, C:\Windows, or C:\Program Files
- Monitor and log file creation in system directories
- Regularly verify **PATH** configuration integrity

DLL Hijacking:

Applications load DLLs from multiple locations. If an attacker can place a malicious DLL in a location searched before the legitimate library, the malicious DLL loads instead.

Attack Scenario: If an application exe01.exe expects dll01.dll from C:\Windows\System32, but an attacker places a malicious dll01.dll in C:\CustomProgram (a directory searched first), the malicious version loads with the application's privileges.

Mitigation:

- Validate DLL integrity using digital signatures
- Use DLL search order safeguards
- Load DLLs from system directories only
- Implement Code Integrity policies

Windows Operating System Vulnerabilities (Cont'd)

1.4 Windows Services and SYSTEM Account

Local System Account (SYSTEM):

- Security Identifier: S-1-5-18
- Most powerful account on a local system
- Services running under SYSTEM have unrestricted system access
- Exploiting SYSTEM-level services grants complete system compromise

Windows Service Control Manager (SCM):

- Launches services using specified service accounts
- Common service accounts: LocalSystem, Local Service, Network Service
- Misconfigured services are prime targets for privilege escalation

Windows Privilege Escalation Exploitation

2.1 Exploitation Workflow

1. **Enumeration:** Identify vulnerable services, permissions, and misconfigurations
2. **Exploitation:** Craft exploits targeting identified weaknesses
3. **Verification:** Confirm privilege escalation success
4. **Post-Exploitation:** Maintain access and gather intelligence

2.2 Enumeration Tools and Commands

PowerShell Commands:

whoami /priv

wmic service get name,pathname,startmode

Get-MpPreference | select DisableRealtimeMonitoring

Third-Party Tools:

- **WinPEAS:** Comprehensive Windows privilege escalation assessment tool
- **Metasploit:** Exploitation framework with built-in Windows exploit modules
- **AccessChk:** Identifies insecure permissions and service configurations

2.3 Known Vulnerabilities (CVEs)

MS17-010 (EternalBlue):

- Affects Windows XP through Windows Server 2016
- Exploits SMBv1 vulnerability
- Allows remote code execution with SYSTEM privileges
- Actively exploited by WannaCry ransomware

MS16-032:

- Secondary Logon Handle Impersonation
- Allows elevation from standard user to SYSTEM
- Affects Windows 7, 8.1, 10, Server 2008-2012 R2

MS15-051:

- Client-Side Caching DLL Privilege Escalation
- Exploits improper handling of cached credentials

Linux Operating System Vulnerabilities

3.1 Overview: Linux systems are prevalent in servers, cloud infrastructure, and IoT devices. While generally more secure than Windows when properly configured, Linux vulnerabilities often stem from misconfigurations rather than OS bugs.

3.2 SUID Binaries

SUID (Set User ID) permission allows an executable to run with the privileges of its owner rather than the executing user.

Vulnerability: If a SUID binary owned by root contains a vulnerability (e.g., a buffer overflow or logic flaw), attackers can execute arbitrary code as root.

Common Vulnerable SUID Binaries:

- cp (copy)
- mv (move)
- chmod (change permissions)
- passwd (change passwords)

List SUID binaries:

find / -perm -4000 -type f -ls 2>/dev/null

Check allowed sudo commands:

sudo -l

Gives us the following:

(ALL) /usr/bin/find, /usr/bin/vim, /usr/bin/awk, /usr/bin/less

Mitigation:

- Remove unnecessary SUID permissions
- Regularly audit SUID binaries
- Keep system patches current
- Monitor SUID binary execution

Linux Operating System Vulnerabilities (Cont'd)

3.3 sudo Misuse and GTFOBins

sudo allows users to execute commands with elevated privileges. Misconfigurations in /etc/sudoers create privilege escalation vectors.

GTFOBins (Get The F* Out of Binaries):** Compilation of tools and binaries that can be exploited for privilege escalation when sudo access is granted. In fact, there is an entire list of such tools; search for GTFOBins in your favourite search engine.

Remote Procedure Call (RPC) Vulnerabilities:

RPC is an interprocess communication mechanism allowing programs to execute code on remote hosts. Several worms exploit RPC vulnerabilities:

- **Conficker Worm:** Exploited RPC vulnerabilities to propagate across networks
- **Detection:** Microsoft Baseline Security Analyzer can identify RPC-related vulnerabilities
- **Pass The Hash:** Vulnerability targeting RPC authentication mechanisms

Vulnerable sudo Configurations:

If a user has sudo access to find, vim, awk, or less, they can execute arbitrary commands:

Use GTFOBins techniques like find:

sudo find . -exec /bin/sh \;

```
$ whoami
lowpriv
$ sudo find /etc/passwd -exec /bin/sh \;
# whoami
root
```

vim:

sudo vim :!/bin/sh

sudo vim -c '!/bin/sh

awk:

sudo awk 'BEGIN {system("/bin/sh")}'

less:

sudo less /etc/passwd

!/bin/sh

Mitigation:

- Use visudo to edit /etc/sudoers safely
- Grant sudo access to specific commands only
- Use NOPASSWD sparingly and only for non-sensitive operations
- Audit sudoers configuration regularly
- Monitor sudo command execution logs

Linux Operating System Vulnerabilities (Cont'd)

3.4 Stored Passwords and Credential Exposure

Plaintext Password Storage:

Administrators sometimes hardcode credentials in shell scripts for automation:

mysql -u admin -p adminpassword

bash_history Exposure:

The ~/.bash_history file contains command history and may expose passwords if an administrator accidentally types them. Common scenario: an admin intends to enter the password interactively but mistypes sudo as sdo and quickly enters it without looking at the screen.

Common Password Vulnerabilities:

- Passwords stored in plaintext in configuration files
- Passwords in script files with improper permissions
- Passwords in command history files
- Default or blank passwords in SQL Server (SQL Server versions prior to 2005 had blank SA passwords by default)

SQL Server Specific Risks:

- Null System Administrator (SA) password is the most critical vulnerability
- SA access through an account with a blank password gives attackers administrative database access

Buffer Overflow Vulnerabilities:

- Data written to the buffer corrupts adjacent memory
- Occurs when copying strings without bound checking
- C and C++ lack built-in protection against memory overwriting
- Can lead to arbitrary code execution with service privileges

Mitigation:

- Use credential vaults (HashiCorp Vault, AWS Secrets Manager)
- Implement environment variables for sensitive data
- Restrict bash_history permissions
- Disable command history for sensitive operations
- Use SSH keys instead of passwords
- Implement password managers for automation
- Enforce strong password policies with a minimum 8 characters
- Require complex passwords (not dictionary words, common terms)
- Implement account lockout thresholds and durations
- Disable LM Hashes in Windows

Embedded Operating System Vulnerabilities

4.1 What Are Embedded Operating Systems?

Embedded Systems:

- Any computer system that isn't a general-purpose PC or server
- Examples: ATMs, GPS devices, routers, smart thermostats, medical devices
- Often critical to infrastructure and daily operations

Embedded Operating Systems:

- Small, efficient programs designed for embedded systems
- Customized for specific functionality
- Examples: Windows Embedded, VxWorks, QNX, RTLinux, OpenWrt

Real-Time Operating Systems (RTOS):

- Guarantee predictable response times
- Used in time-critical applications: aerospace, medical devices, industrial control

4.2 Embedded OS Types and Examples

Windows Embedded:

- Windows Embedded Standard (based on Windows 7)
- Windows Embedded Enterprise
- Windows IoT

Proprietary Embedded OSs:

- **VxWorks:** Used in aerospace, defense, and medical devices
- **Green Hills Software:** F-35 Joint Strike Fighter, routers, switches
- **QNX:** Cisco routers, Logitech remotes
- **RTEMS:** Open-source, used in space systems

Linux-Based Embedded:

- **Embedded Linux:** Industrial, medical, consumer devices
- **RTLinux:** Monolithic kernel extension for real-time requirements
- **OpenWrt/dd-wrt:** Networking devices (Linksys WRT54G routers)

Embedded Operating System Vulnerabilities (Cont'd)

4.3 Critical Vulnerabilities in Embedded Systems (Cont'd)

Network Peripherals as Attack Vectors

Multifunction Devices (MFDs) - printers, scanners, copiers, fax machines:

Vulnerabilities:

- Rarely scanned for security vulnerabilities
- Often not configured for security
- Contain stored sensitive information (print jobs, scanned documents)
- Information susceptible to theft and modification
- May store credentials for network access

Exploitation:

- Extract stored print jobs to obtain confidential information
- Use a compromised device as a gateway into secure networks
- Alter printer output to inject malicious links targeting user desktops
- Social engineering techniques to gain administrative access

Cell Phones and Smartphones Vulnerabilities:

- Attackers listening to phone calls (if using vulnerable protocols)
- Using phone as a microphone for eavesdropping
- Phone cloning to make fraudulent calls
- Extracting information for computer/network access
- Stealing trade secrets or national security information
- Java-based phone viruses
- Cell phone rootkits for persistent compromise

Embedded Operating System Vulnerabilities (Cont'd)

4.4 Rootkits in Embedded Systems

Rootkit Definition: Malicious software that modifies OS parts or installs as kernel modules, drivers, libraries, or applications.
Rootkit Persistence: Survive system reboots and evade detection mechanisms.
Embedded System Risk: Embedded devices may be compromised during manufacturing before reaching customers. Firmware-level infections are particularly dangerous.

Detection Difficulty: Rootkits that control the OS kernel can evade detection by traditional tools.

Remediation:

- Flash (rewrite) BIOS/firmware to remove kernel-level malware
- Wipe the hard drive completely
- Reload a clean operating system
- Time-consuming and expensive process

Decision Workflow for Handling a Vulnerability

(Figure 3)

Source: Gartner ID: 410271

Best Practices for Hardening Operating Systems

5.1 Windows Hardening

Patch Management:

- Deploy security updates promptly using Windows Update or WSUS
- For large networks: use Systems Management Server (SMS), Windows Software Update Service (WSUS), or SCCM
- Third-party solutions: BigFix, Tanium, BladeLogic
- Keep systems updated as attackers exploit known vulnerabilities

Antivirus Solutions:

- Essential for all Windows systems
- Small networks: desktop antivirus with automatic updates
- Large networks: corporate-level solution required
- Update antivirus definitions regularly (tools are useless if not updated)

User Account Control (UAC):

- Enable and maintain UAC settings
- Educate users on prompt approval

Firewall Configuration:

- Enable Windows Defender Firewall
- Implement host-based intrusion detection
- Monitor inbound/outbound connections
- Filter ports 137-139 and 445 (against SMB and null session attacks)

Service Hardening:

- Disable unnecessary services
- Run services with least privilege accounts
- Regularly audit service configurations
- Remove unused scripts and sample applications
- Delete default hidden shares

File System Security:

- Enable NTFS encryption (EFS) for sensitive data
- Implement BitLocker full disk encryption
- Audit and maintain file permissions
- Use a file integrity checker like Tripwire

Network Security:

- Implement network segmentation
- Use appropriate packet filtering with firewalls and Intrusion Detection Systems
- Limit administrator accounts
- Use different naming schemes and passwords for public interfaces
- Ensure password length and complexity requirements

Account Security:

- Disable the Guest account
- Disable the local Administrator account
- Ensure no accounts have blank passwords
- Implement strong password policies

Access Control:

- AppLocker and Code Integrity for application whitelisting
- Enforce code signing policies
- Use Windows Group Policies for security enforcement

Monitoring and Logging:

- Enable and review logs regularly
- Monitor critical areas and traffic patterns
- Identify signs of intrusion and problems

Best Practices for Hardening Operating Systems (Cont'd)

5.2 Linux Hardening

Kernel and Package Updates:

- Apply security patches immediately upon release
- Subscribe to security advisory lists for your distribution
- Use automated patching where possible
- Most Linux distributions have warning methods for critical vulnerabilities

Access Control:

- Implement the principle of least privilege
- Use role-based access control (RBAC)
- Restrict SUID binaries to only necessary programs
- Configure sudoers carefully using visudo

Firewall Configuration:

- Implement iptables/firewalld rules
- Enable SELinux or AppArmor mandatory access controls
- Monitor firewall logs

Service Hardening:

- Disable unnecessary services and daemons
- Run services as unprivileged users
- Use systemd security features

File System Security:

- Implement disk encryption (LUKS)
- Set appropriate file permissions
- Monitor file integrity with AIDE or Tripwire
- Be cautious with the bash_history file permissions

User Awareness Training:

- Inform users not to share OS information with outsiders
- Users should be suspicious of people asking questions
- Educate on verifying identities before sharing information

Vulnerability Assessment:

- Use vulnerability scanners regularly
- Use built-in Linux tools for security assessment
- Implement free benchmark tools from the Center for Internet Security

Best Practices for Hardening Operating Systems (Cont'd)

5.3 Embedded System Hardening

Firmware Management:

- Keep firmware current with the latest security patches
- Verify firmware authenticity and integrity
- Use secure boot mechanisms

Network Security:

- Change default credentials immediately
- Disable unnecessary network services
- Implement network segmentation
- Use VPNs for remote management access

Data Protection:

- Encrypt data in transit (TLS/SSL)
- Encrypt sensitive data at rest
- Secure credential storage

Monitoring and Logging:

- Enable logging for administrative access
- Monitor for unusual device behavior
- Maintain centralized log collection
- Alert on suspicious activities

Lifecycle Management:

- Identify all embedded systems in the organization
- Prioritize systems by function criticality
- Plan for end-of-life replacement
- Upgrade or replace systems that cannot be secured

Detection and Remediation

6.1 Rootkit Detection and Removal

Rootkit Characteristics:

- Modify OS kernel modules, drivers, libraries, or install as applications
- Exist for Windows, Linux, and embedded systems
- Provide persistent backdoor access across reboots
- Can be installed at the factory level, compromising devices before sale

Detection Methods:

- Behavioral analysis and monitoring
- Kernel integrity verification
- Memory forensics
- File system analysis
- Rootkit-detection tools and antivirus software

Detection Tools:

- chkrootkit
- rkhunter
- GMER
- Windows Defender offline scanning

Limitations: If the OS is already compromised, detection tools may be unreliable, as rootkits can hide their presence.

Remediation:

- Flash (rewrite) BIOS/firmware to remove kernel-level malware
- Wipe the hard drive completely
- Reload a clean operating system
- Complete system replacement if firmware-level infection suspected
- Time-consuming and expensive process

6.2 Windows Defender Exploit Guard (Windows 10+)

New Security Features (Fall Creators Update onwards):

- **Attack Surface Reduction (ASR):** Uses machine learning to detect and stop zero-day attacks
- **Network Protection:** Blocks outbound connections to known malicious servers
- **Controlled Folder Access:** Prevents unauthorized programs from modifying files in protected folders
- **Exploit Protection:** Guards against memory corruption and code injection attacks

6.3 Incident Response

Upon Suspected Compromise:

1. Isolate the system from the network
2. Preserve evidence for forensic analysis
3. Conduct a memory dump for analysis
4. Perform comprehensive malware scanning
5. If rootkit confirmed: complete OS wipe and reload, or hardware replacement

Ethical Considerations

As ethical hackers, understanding these vulnerabilities enables you to:

- Identify security flaws before malicious actors exploit them
- Help organizations implement protective measures
- Conduct authorized penetration tests to improve security posture
- Educate stakeholders about risks and mitigations

Always:

- Obtain written authorization before testing any system
- Follow responsible disclosure practices
- Respect legal and ethical boundaries
- Document findings professionally

References

Here are some helpful online resources. You can also find more resources from the MMU digital library:

- Windows Internals (7th Edition), Part 2: Chapter 10 - Windows Services
- NIST Cybersecurity Framework:
  - https://www.nist.gov/cyberframework
- Microsoft Security Advisories:
  - https://learn.microsoft.com/en-us/security-updates
  - https://msrc.microsoft.com/update-guide
- Linux Foundation Security Hardening Guidelines:
  - https://www.linuxfoundation.org/lf-security
- OWASP IoT Top 10 and IoT Security Resources
  - https://owasp.org/www-chapter-toronto/assets/slides/2019-12-11-OWASP-IoT-Top-10---Introduction-and-Root-Causes.pdf

Summary

- Operating systems (Windows, Linux, Embedded) are vulnerable in different ways.
- Many real-world attacks exploit **misconfigurations**, not just CVEs.
- Embedded devices are critical yet under-secured and hard to patch.
- Ethical hackers must:
  - Identify flaws before attackers do.
  - Educate on risks and mitigations.
  - Apply responsible and legal testing.

Thank You

ANY QUESTIONS?